LAB ASSIGNMENT

NLP Foundations: Handwriting OCR Evaluation

Deepening your understanding after Lab 6

Course	Artificial Intelligence
Practical	Lab 6 of 8 - Handwriting OCR Evaluation
Student details	Name: Muhammad Saqib Student ID: 24009307

Purpose

This assignment is to be completed after the Lab 6 practical. You do not need to repeat the entire experiment. Instead, use what you learned to explain the concepts, interpret the results, reason about OCR errors and suggest improvements. Answer in your own words.

Instructions

1. Complete the assignment individually.
2. Use your Lab 6 practical outputs and the results discussed during the lab as evidence.
3. Show all calculation steps. Round numerical answers to 2–3 decimal places where appropriate.
4. You may refer to your lab results; you do not need to rerun the complete OCR experiment.
5. Include screenshots or small tables only where requested. Ensure that all evidence is readable.

Part A - Understand the concepts

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

Example: Photo of a handwritten prescription containing the medicine name: Amoxicillin 500mg

Stage 1. Acquire: This stage involves collecting raw data needed for the pipeline. It covers sourcing the data, gathering it, and making sure it is stored in a usable form. **Example:** A pharmacy app captured a photo of a doctor's prescription using a camera. For training, the team might also collect thousands of handwritten samples of medicine names.

Stage 2. Normalize: This stage involves cleaning the raw data and organizing it into a consistent and standard form so noise and variation don't affect later stages. For typed text, this includes lowercasing, removing punctuation, fixing spelling, and tokenizing. **Example:** The image is processed by converting it into greyscale, removing shadows and paper texture, correcting skew, and sharpening faint strokes. An OCR or handwriting recognition engine then converts the image into text.

Stage 3. Represent: This stage involves converting raw text into numerical forms. Common choices include bag-of-words, TF-IDF, character n-grams, and word or subword embeddings. **Example:** The cleaned token is turned into a vector. Character n-grams work well for drug names because they stay robust when a character is misread.

Stage 4. Model: This phase involves training an algorithm or applying it to learn patterns from the representations and make predictions. Depending on the task, this could be classification, sequence labelling, translation, or generation. **Example:** A classifier or sequence model takes the vectors and predicts which medicine from a known formulary is being written. It might output "Amoxicillin (95%), Ampicillin (3%), Amoxapine (1%)." A named-entity recognition model could also tag 500mg as a dosage and amoxicillin as a drug name. In practice a CNN-LSTM or transformer-based handwriting recognizer is often used end to end.

Stage 5. Evaluate: This stage measures how well the system performs using suitable metrics and test data the model has never seen, then feeds the findings back into earlier stages for improvement. **Example:** The team tests on a held-out set of prescriptions and measures character error rate, word accuracy, and precision and recall per medicine. Because mistakes here are safety-critical (confusing "Hydralazine" with "Hydroxyzine" could harm a patient), they pay special attention to look-alike drug names and set a confidence threshold so uncertain predictions get flagged for a pharmacist to review. Error analysis may show that messy "m" and "n" strokes cause most failures, which sends the team back to improve normalization or collect more training data.

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Segmentation answers the question **"where are the units?"** It divides the image into pieces that can be recognised one at a time: first the page into lines, then lines into words, and often words into individual characters or strokes. It deals with geometry and boundaries, not meaning. It decides which pixels belong together.

Example failure: In a cursive handwritten "Amoxicillin", the letters are joined by connecting strokes, and the "m" has three humps. The segmenter wrongly splits the "m" into two pieces and treats the double "l" as a single blob. The downstream recogniser now receives fragments that don't correspond to real letters.

Character recognition answers the question **"what is this unit?"** Given an isolated image patch, it classifies it as a particular letter or digit, usually by outputting probabilities across the alphabet. It deals with identity and shape, not boundaries.

Example failure: The segmenter correctly isolates a cleanly separated character in "Hydralazine", but the recogniser misreads a sloppy "a" as an "o" or a "u". The boundary was right; the label was wrong. The result might be "Hydrolozine", or in the worst case a different real drug name.

Q3. Text normalization

The reference is “Tab Azimax 40mg” and the OCR output is “tab azimax 40 mg”. Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

	Original	Lowercase	Letters and Digits Only
Reference	Tab Azimax 40mg	tab azimax 40mg	tabazimax40mg
OCR Output	tab azimax 40 mg	tab azimax 40 mg	Tabazimax40mg

The reason that the same rule must be applied to both is that it ensures that the results are consistent and reproducible.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

Removing spaces can hide an important difference when a space is the only thing separating two numbers. For example, suppose the reference is “Warfarin 5 2 mg” and the OCR output is “Warfarin 52 mg”. The Lab 6 rule turns both into “warfarin52mg”, so the evaluation reports a perfect match. But one string has two separate numbers and the other has a single dose of 52 mg, which is a serious dosing error that the metric cannot see.

Part B — Work with OCR measurements

Q5. Levenshtein distance

Calculate the Levenshtein distance between “breeky” (reference) and “beeky” (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

The Levenshtein distance is 1.

The reference “breeky” has 6 letters (b, r, e, e, k, y) and the prediction “beeky” has 5 letters (b, e, e, k, y). The prediction is missing the “r” after the first “b”. Deleting that “r” from the reference gives “beeky”, so one edit is enough to turn one string into the other. It cannot be zero because the strings are different, so 1 is the minimum.

The edit operation is a single deletion: the “r” at position 2 of the reference is missing from the prediction. (Viewed the other way, turning the prediction into the reference would be one insertion of “r”.)

The normalised edit distance is the distance divided by the length of the longer string. The longer string is “breeky” with 6 characters, so $1 / 6 \approx 0.167$, or about 16.7%.

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

ER = (S + D + I) / N = (4 + 2 + 1) / 35 = 7 / 35 = 0.2, or 20%.

This means the OCR system made errors equal to 20% of the reference length. Put another way, you would need 7 character edits to turn the OCR output into the reference, and about 1 in every 5 characters is wrong. A lower CER is better, and 0 means a perfect match.

Yes, CER can be greater than 1. The denominator N is only the length of the reference, but insertions are not limited by it. If the OCR output is full of extra characters, the total number of errors can exceed N. For example, if the reference has 10 characters and the system outputs 25 characters of garbage, the errors could be 20 or more, giving a CER of 2.0 or higher. This is one reason CER should be read alongside other measures, since a value above 1 means the output is worse than producing nothing at all (which would give a CER of exactly 1, since every character would be a deletion).

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Exact-match accuracy is the share of predictions that are identical to the reference, usually after normalisation. Each prediction is simply right or wrong, with no partial credit. For example, if 60 out of 100 medicine names match exactly, the accuracy is 60%. Higher is better.

Mean normalised edit distance takes the Levenshtein distance between each prediction and its reference, divides it by the length of the longer string, and averages this over all samples. Each sample scores between 0 and 1, where 0 is a perfect match and 1 is completely different, so the average tells you how close predictions are in general, even when they are not exactly right. Lower is better.

CER (character error rate) is the total number of substitutions, deletions and insertions across the dataset divided by the total number of characters in the references. It measures how many individual characters the system gets wrong. Unlike normalised edit distance, it divides by the reference length, which is why it can go above 1 when there are many insertions. Lower is better.

Blank predictions are cases where the system outputs an empty string, meaning it found nothing to read. This is usually reported as a count or percentage of samples. A blank is a distinct kind of failure from a misread: the system gave no answer at all, and in practice this may need different handling, such as sending the image to a human. Each blank contributes a CER of exactly 1 for that sample. Lower is better, and ideally it is zero.

Why exact-match accuracy alone is insufficient: it treats every failure the same. “Amoxicilin” for “amoxicillin” (one missing letter) counts as wrong in exactly the same way as a blank output or random characters. So two systems with the same exact-match score could be very different, with one nearly right on every miss and the other completely off. Exact match also gives no clue about what to fix. It cannot show whether errors are small slips, which might be solved with a dictionary correction step, or total failures, which point to problems earlier in the pipeline. Edit distance and CER capture how large the errors are, and the blank count shows how often the system produces nothing, so together they give a much more useful picture.

Part C — Interpret the Lab 6 experiment

Use the results from Lab 6. Both pipelines were evaluated on the same 89 labelled images, with the same normalisation rule and fixed Tesseract configuration.

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

1. Far fewer blank predictions. Blanks dropped from 42 to 15 out of 89 images. That means the baseline returned nothing for about 47% of the images (42/89), while the preprocessed pipeline did so for about 17% (15/89). Preprocessing helped Tesseract find and read text in 27 more images, which is the biggest gain in the table.

2. Lower error rates. The character error rate fell from 0.883 to 0.806 (about 7.7 percentage points), and the mean normalised edit distance fell from 0.869 to 0.785 (about 8.4 points). Since lower is better for both, the pre-processed outputs are on average closer to the references, even when they are not exactly right.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Exact-match accuracy only rewards a prediction that is perfect, whereas the other metrics reward partial progress. Preprocessing mostly moved images from “nothing read” to “something read, but still wrong”, and that kind of progress shows up in CER, edit distance and blanks but barely in exact match.

Consider what happened to the 27 images that stopped being blank. A blank prediction scores the worst possible result for that image (CER of 1, normalised edit distance of 1). Once preprocessing helps Tesseract detect some text, even a garbled output such as “amoxicilin” or “amoxlcllin” is much closer to the reference than an empty string, so the error metrics improve. But a medicine name is only counted as correct if every character matches, and handwriting is hard for Tesseract. One wrong letter, a missed character or a stray symbol is enough to fail exact match. So an image can go from fully wrong to mostly right and still count as a failure.

Preprocessing also has limits in what it can fix. Converting to grayscale, adding a border and resizing make the image easier to detect and read, but they do not change how Tesseract recognises handwriting. The recogniser is trained mainly on printed text, so it still confuses similar-looking letters and mishandles cursive. Preprocessing helps the earlier steps (finding text, reading something) more than the final accuracy of each character.

The numbers fit this. Blanks fell from 42 to 15, and CER fell only from 0.883 to 0.806, so most of the remaining output is still wrong at the character level. With about 8 in 10 characters still incorrect, it is unsurprising that only 1 of 89 names is fully correct.

This is why exact match alone would have made preprocessing look almost useless (0.000 to 0.011), while the other metrics show that it did help. Preprocessing improved recovery of text, but not yet the accuracy needed to get whole medicine names right.

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

The lab kept those three things fixed so that the only difference between the two pipelines was the preprocessing (grayscale, border and resize). That makes it a controlled experiment: if the results change, the change can be attributed to preprocessing and nothing else. If several settings changed at once, it would be difficult to conclude what caused the improvement.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;

The system never produced a prediction at all. The cause is usually technical, such as a corrupt or unreadable image file, an unsupported format, a wrong path, or a bad configuration. It is different from OCR failing to read the handwriting. In evaluation it should be counted and reported separately, or the image excluded with a note, because there is no output to compare with the reference. The fix is to check the file and the pipeline code, not to improve the recogniser.

(b) OCR returns an empty string;

Tesseract ran without crashing but found nothing it was willing to read. Typical causes are low contrast, faint strokes, text too small or too close to the image edge, or handwriting too unlike what it was trained on. This is the failure that preprocessing helped most in the lab, since blanks fell from 42 to 15. Each blank scores a CER of 1 for that image.

(c) OCR returns “nidu” when the reference is “nims”.

The prediction has the same length as the reference, and the first two letters match. Only the last two characters are wrong: “m” was read as “d” and “s” as “u”. Because the number of characters is correct, this is more likely a misread of letter shapes than a segmentation problem, where characters would be merged, split, or missing. The edit distance is 2 (two substitutions), so the normalised distance is 2/4 = 0.5. Fixes would involve better preprocessing of the stroke shapes, a recogniser better suited to handwriting, or a drug-name dictionary check.

Part D — Apply your understanding

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

1. Adaptive thresholding and contrast enhancement (targets blank predictions and stroke misreads).

The lab's preprocessing (grayscale, border, resize) cut blanks from 42 to 15, but 15 images still return nothing, and the CER of 0.806 suggests many strokes are still being misread. Faint pen strokes, uneven lighting and paper texture are plausible causes. Adaptive thresholding (for example Otsu or local thresholding) and contrast enhancement could make strokes clearer, so Tesseract detects more text and misreads fewer letters.

To test it, I would add the new steps as a third pipeline and keep everything else identical: the same 89 images, the same --oem 1 --psm 7 settings and the same normalisation rule. Then I would compare it with grayscale_border_resize on blanks, CER and mean normalised edit distance. It counts as helping only if blanks drop below 15 and CER falls below 0.806 by more than a trivial amount. Because there are only 89 images, I would also check per-image results in predictions.csv to see whether the gain is spread across many images or comes from a few, and I would look at whether any images got worse. If the thresholding makes some images worse, I would report that rather than only the average.

2. Dictionary-based post-correction (targets near-miss recognition errors).

Outputs like "nidu" for "nims" or "amoxicilin" for "amoxicillin" are close to a real medicine name but not exact. If the system had a list of valid medicine names, it could replace each raw prediction with the nearest entry by edit distance. This targets character recognition errors, and it is what could move the exact-match accuracy, which is currently only 0.011.

To test it, I would apply the correction to the existing predictions from the preprocessed pipeline, so that the OCR itself is unchanged and only the new step differs. Then I would compare exact-match accuracy, CER and normalised edit distance before and after. The main risk is that correction can turn a near miss into a confidently wrong drug, which is dangerous with look-alike names. So I would also count how many corrected predictions are wrong, and how many were right before and wrong after. I would also set a distance threshold, so that predictions too far from any dictionary entry are flagged for human review instead of being forced to a match. Since a dictionary built from the same 89 reference names would give an unfairly good score, the list should be larger than, or separate from, the test labels.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

1. Wrong drug names that look plausible. OCR errors are often near misses, like "nidu" for "nims". In medicine, a near miss can be a different real drug. Look-alike pairs such as hydralazine and hydroxyzine differ by only a few characters, so a small recognition error, or an over-eager dictionary correction, can turn one into the other. A wrong drug dispensed to a patient can cause serious harm.

2. Dosage errors that look correct. Numbers and units are short, so one misread digit or a merged or split number changes the meaning completely. As we saw with "5 2 mg" versus "52 mg", a lenient normaliser or a careless reader can hide a tenfold error. A dose error can cause an overdose or an ineffective treatment, and the output looks just as tidy as a correct one.

3. Silent failures and false confidence. Blank predictions, crashes and garbled output can pass unnoticed if nobody reviews them. In the lab, about 17% of images still came back blank even after preprocessing. If the system quietly skips these, a prescription may go unfilled or be filled from incomplete data. Staff may also

over-trust the system because the output is typed and looks authoritative, which reduces the chance that anyone spots a mistake.

A further risk is accountability and privacy. If an error occurs, it may be unclear who is responsible, and prescription images contain sensitive patient data that must be stored and processed securely.

A safer workflow

1. **Capture carefully.** Use good lighting and a standard way of photographing the prescription, and reject images that are too blurry or dark.
2. **Run OCR as a suggestion only.** Treat the output as a draft that assists staff, never as the final record.
3. **Check against a drug database.** Match the drug name and dose against an approved formulary, and flag anything that is not a valid drug, strength or dose combination. Flag, do not auto-correct.
4. **Flag uncertain cases.** Send blanks, errors, low-confidence outputs and any mismatch with the database straight to a person.
5. **Require pharmacist verification.** A pharmacist compares the OCR result against the original prescription image, ideally shown side by side, and confirms the drug, strength, dose and frequency before anything is dispensed. For high-risk drugs, a second check or a call to the prescriber is wise.
6. **Log and monitor.** Record errors and corrections, track metrics such as CER and blank rate over time, and use the corrected cases to improve the system. Only consider reducing checks if evidence from real use shows consistently low and safe error rates, and even then keep human review for high-risk cases.

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

Submission to your AI Lab Git repository

IMPORTANT: Submit your completed assignment to the Git repository you created for the AI Lab.

Do not keep the assignment only on your computer or send it only through a private message. Upload it to your existing AI Lab Git repository.

6. Create a folder named Lab06_Assignment.
7. Save your completed report as Lab06_OCR_Assignment_YourName.pdf (or .docx).
8. Put supporting screenshots or figures in a subfolder named evidence.
9. Commit and push your changes to your existing AI Lab repository(if not done yet).
10. Open the repository online and confirm that your files are visible.

Suggested repository structure

```
AI-Lab-Repository/
└── Lab06_Assignment/
```

```
├── Lab06_OCR_Assignment_YourName.pdf
└── evidence/
    ├── example_1.png
    └── results_table.png
```

Assessment focus

Your work will be assessed on conceptual understanding, correct calculations, interpretation of the actual Lab 6 results, quality of reasoning, clarity of writing and completeness of the Git repository submission.

I learnt that a single metric can hide progress, so a system has to be judged with several metrics together. In Lab 6, exact-match accuracy only moved from 0.000 to 0.011, which makes preprocessing look nearly useless. But blanks fell from 42 to 15, CER fell from 0.883 to 0.806, and mean normalized edit distance fell from 0.869 to 0.785. Preprocessing helped Tesseract read something in many more images, even though almost none of the names became fully correct. Reading only exact match would have led to the wrong conclusion.

Final reminder

The purpose is understanding, not simply reproducing code or copying results. Explain what the results mean, why the experiment was designed this way and what you would investigate next.